TaxTrace

Reading pack

Deloitte, Analyst Tax Data Automation
Seven day preparation

Contents

TaxTrace

Configurable tax determination, validation and reconciliation engine.

Takes messy multi-ERP finance data, applies tax rules held as configuration rather than code, calculates expected tax, reconciles it against what the source system recorded, classifies every variance, and leaves an audit trail from any filed figure back to the source row and the rule version that produced it.

Built against a synthetic but deliberately realistic client, Meridian Commerce Group, operating across Australia, New Zealand, the United Kingdom and eight United States states.

Why the trail matters

Most tax data tooling can tell you *what* the tax should be. The harder question, and the one an auditor actually asks, is *why*. Every calculated row in TaxTrace carries the rule id, the rule version, the rate, and the timestamp that produced it. Given a filed number you can name the rule that made it.

Status

Stage	State
Business requirements	Complete
Synthetic data generator with known ground truth	Complete, tested
Data profiling	Day 1, in progress
Canonical model and SQL layer	Day 2
Rules engine	Day 3
Data quality and reconciliation	Day 4
Regression suite and incident response	Day 5
CI/CD, documentation, AI evaluation harness	Day 6

Quick start

```
git clone <repo>
cd taxtrace
python -m venv .venv && .venv\Scripts\activate     # Windows
pip install -r requirements.txt

python scripts/generate_data.py          # writes data/, about 18 MB
python -m pytest -q                      # 20 tests
```

The dataset is **not committed**. It is regenerated deterministically from a seed, which is why `data/` is in `.gitignore`. Same seed, same 50,300 rows, same 12,468 injected faults, every time.

The dataset

Four ERP extracts, each using its own field names, boolean conventions and rate formats, exactly as a real multi-system client would send them.

File	Rows	Field naming
erp_sap_au_transactions.csv	18,182	SAP (`BELNR`, `MWSKZ`, `WMWST`)
erp_dynamics_uk_transactions.csv	14,061	Dynamics (`DocumentNo`, `VATAmount`)
erp_oracle_us_transactions.csv	12,076	Oracle (`TRX_NUMBER`, `TAX_AMT`)
erp_netsuite_nz_transactions.csv	5,981	NetSuite (`tranid`, `taxtotal`)

Plus customers, vendors, jurisdictions, tax codes, an effective-dated rate history, and a 250-row golden dataset.

Injected faults

12,468 faults across 20 fault types, logged by transaction id in an answer key so a pipeline's precision and recall can actually be measured.

Tier	Count	Findable by
LOUD	6,025	Profiling
MEDIUM	2,792	Referential integrity and business rules
SUBTLE	3,651	Reconciliation and statistics only

The SUBTLE tier is the interesting one: tax understated by a fraction of a per cent, banker's rounding where half-up was required, an exemption flag set without a valid certificate, a gross amount reported as net.

Architecture

```
4 ERP extracts → ingest → profile → stage → map to canonical → clean
   → data quality gate → rules engine → calculate → validate against golden
   → reconcile → classify variance → detect anomalies → exception register
```

Full detail, including how each stage would map onto Azure Data Factory, Synapse and Airflow in an enterprise build, is in `docs/architecture.md`.

Documentation

All documentation is also available as PDF in `docs/pdf/`, including a single combined **reading pack** (24 pages, with contents page). Regenerate with:

```
python scripts/md_to_pdf.py
```

Document	Contents
`docs/business_requirements.md`	Client objective, 20 functional requirements, risks, acceptance criteria
`docs/architecture.md`	Pipeline, layer contracts, cloud mapping, ETL vs ELT
`docs/skill_matrix.md`	Role requirements mapped to project evidence
`docs/study_plan.md`	Seven day plan
`docs/day01.md`	Day 1 concepts and exercise

Tech

Python, pandas, Polars, DuckDB, SQL, pytest, YAML configuration, Git, GitHub Actions. Runs entirely locally. No paid services, no cloud account required.

Key engineering decisions

---

1. **Ground truth first, corruption second.** The generator computes the

correct tax, stores it, then damages the source data. Every exception the pipeline raises is therefore checkable against a known answer.

1. **Rules as configuration.** A tax analyst changes a YAML decision table. No

Python change, no deployment, and the regression suite proves nothing else moved.

1. **Effective dating throughout.** Rates are resolved as at the transaction

date, never today's date. Repricing history with a current rate is one of the most expensive bugs in tax data.

1. **Half-up rounding, explicitly.** Python's `round()` is banker's rounding.

Finance is not. The generator, the engine and the tests all use `Decimal(ROUND_HALF_UP)`.

1. **Raw data is immutable.** Everything is re-derivable from the landing zone, which is what makes a six-month-late rate correction possible.

Deloitte Analyst, Tax Data Automation – skill matrix

Requisition 41834, Melbourne. Derived from the advertised responsibilities and requirements, then ranked by how much of the interview is likely to sit on each.

The job in one sentence

Take a client's messy ERP finance data, turn the tax team's rules into configuration a machine can execute, and prove the answer is right, repeatably, with a trail an auditor can follow.

Full requirement mapping

#	Job requirement	Category	Weight	Where the project covers it
1	Python for ETL/ELT processing	Core	High	Days 1, 2, 4
2	SQL over relational data	Core	High	Day 2, DuckDB model
3	Translate tax requirements into executable rules	Core	High	Day 3 rules engine
4	Tax calculation, classification, validation rules	Core	High	Day 3
5	Automated data quality controls	Core	High	Day 4 DQ framework
6	Reconciliation systems	Core	High	Day 4
7	Data validation	Core	High	Days 1, 4
8	Diagnose production issues in tax modules	Core	High	Day 5 incident
9	Automated testing, regression	Core	High	Day 5 pytest suite
10	Git, code review, CI/CD	Core	Medium	Day 6
11	Data workflows, schemas and mappings	Core	High	Day 2 canonical model
12	ERP and finance data	Domain	High	4 source extracts
13	Requirements workshops with Tax SMEs	Soft	High	Day 3 + Day 7 simulation
14	Document technical decisions and rules	Soft	Medium	Throughout, docs/
15	Pandas, Polars, DuckDB	Tools	Medium	Day 2 benchmark
16	Rules engines, decision tables	Concept	High	Day 3

#	Job requirement	Category	Weight	Where the project covers it
17	Configuration driven processing	Concept	High	Day 3 YAML
18	Reference / golden datasets	Concept	Medium	Day 4
19	Output comparison, regression testing	Concept	Medium	Day 5
20	Anomaly detection	Concept	Medium	Day 4
21	AI assisted engineering, LLM evaluation	Emerging	Medium	Day 6 eval harness
22	Agentic workflows	Emerging	Low	Day 6, conceptual
23	Azure, Docker, Kubernetes, Airflow	Cloud	Low	Day 6, conceptual only
24	Statistical anomaly detection	Advantageous	Low	Day 4

The 13 skills to actually master this week

Ranked. If you run out of time, cut from the bottom.

1. **Rules as configuration, not code.** The single most distinguishing idea in this job. A Tax SME must be able to change a rate without a developer.
1. **Reconciliation and variance classification.** Source tax versus calculated tax, and being able to say *why* they differ.
1. **SQL for investigation.** CTEs, window functions, duplicate detection, conditional aggregation. This is what a live technical screen will test.
1. **Data quality controls with severity and thresholds.** Named, versioned, measurable checks. Not ad hoc asserts.
1. **Effective dating and rule precedence.** Which rate applied *on that date*.
2. **Canonical schema and source mapping.** Four ERPs, one model.
3. **Rounding, tax-inclusive versus exclusive, currency.** The arithmetic that quietly generates thousands of one cent variances.
1. **Pytest and regression testing of business logic.** Change a rule, prove nothing else moved.
1. **Production incident diagnosis.** Symptom to root cause to fix to verification to written incident report.
1. **Traceability.** Every calculated number carries the rule id and version that produced it. This is the audit requirement and your differentiator.

1. **Pandas versus Polars versus DuckDB.** When and why, with a measured

benchmark rather than an opinion.

1. **Git discipline.** Branches, meaningful commits, a PR you would approve.
2. **AI evaluation harness.** Natural language rule to structured proposal to

deterministic validation to human approval.

Where your existing background already maps

You are not starting from zero on the domain half, which is the half most candidates for this role are weakest on.

Your experience	Job requirement it maps to
Stripe tax operations, multi state US filings	ERP and finance data, tax compliance context
Correcting tax refunds and tax data	Reconciliation, exception handling
Addresses affecting tax calculation	Jurisdiction determination, the hardest part of sales tax
Escalation team for complex cases	Production incident triage
US stakeholders	Requirements gathering with SMEs
Excel macro, 20 per cent time saved	Automation, process improvement
Master of Data Science	Python, SQL, statistics, anomaly detection

The gap is not tax knowledge and it is not analysis. The gap is **engineering practice**: configuration driven design, automated tests, version control discipline, and being able to prove correctness rather than assert it. That is what the next seven days are for.

Business Requirements – TaxTrace

Client: Meridian Commerce Group **Engagement:** Tax Data Automation, indirect tax validation and reconciliation **Prepared by:** Tax Transformation and Technology **Status:** Baseline, v1.0

1. Client background

Meridian Commerce Group is a mid-market retail and e-commerce group operating across Australia, New Zealand, the United Kingdom and eight United States states. Annual revenue is approximately AUD 480 million.

Growth by acquisition has left Meridian with four finance systems that were never consolidated.

Source system	Region	Approx. share of transactions
SAP ECC	Australia	36%
Microsoft Dynamics 365	United Kingdom	28%
Oracle Fusion	United States	24%
NetSuite	New Zealand	12%

Each system holds the same concepts under different field names, different boolean conventions, and different date formats. Tax rates are configured independently in each, by different people, at different times.

2. The problem

Indirect tax returns are prepared monthly in Excel. A senior analyst downloads extracts from each ERP, joins them by hand, eyeballs the totals against last month, and files. The process takes nine working days.

Three incidents in the last eighteen months have driven this engagement.

- A UK VAT rate change was applied in Dynamics two weeks late. 4,100

transactions were filed at the wrong rate. The error was found by the external auditor, not by Meridian.

- An expired US resale exemption certificate continued to be honoured for

seven months, understating collected sales tax.

- A duplicate invoice feed from NetSuite double counted NZD 210,000 of output

tax. It was found only because a customer queried their statement.

None of these were found by a control. All three were found by an outsider.

3. Business objective

> *Automatically determine, validate and reconcile indirect tax on every transaction before filing, so that errors are found by Meridian's own controls rather than by an auditor or a customer, and so that every calculated figure can be traced back to the source record and the rule that produced it.*

4. Users

User	What they need
Indirect Tax Manager	Confidence the filing is right, and evidence for why
Tax Analyst	To change a rate or a treatment without a developer
Finance Operations	A clear, categorised exception list to work through
Data Engineering	Reproducible, testable, version controlled transformations
Internal Audit	A traceable line from filed figure back to source row and rule
External Auditor	Documented rules, documented controls, evidence of testing

5. Functional requirements

ID	Requirement	Priority
FR-01	Ingest transaction extracts from all four source systems	Must
FR-02	Map each source schema onto a single canonical transaction model	Must
FR-03	Profile incoming data and report quality before processing	Must
FR-04	Standardise country, currency, region, category and date values	Must
FR-05	Determine the applicable tax code from jurisdiction, product category, customer status and transaction date	Must
FR-06	Apply the rate effective **on the transaction date**, not today's rate	Must
FR-07	Support tax-inclusive and tax-exclusive source amounts	Must
FR-08	Honour customer exemption certificates only within their validity dates	Must
FR-09	Allow a Tax Analyst to add or change a rule in configuration, with no code change	Must
FR-10	Record rule id and rule version against every calculated result	Must
FR-11	Reconcile calculated tax against source tax and classify each variance	Must
FR-12	Run a named, versioned set of data quality checks with severities	Must
FR-13	Produce an exception report categorised by cause and owner	Must

ID	Requirement	Priority
FR-14	Compare output against a golden dataset and report pass, fail or warning	Must
FR-15	Detect statistical anomalies in tax rate, tax amount and volume	Should
FR-16	Detect duplicate transactions and duplicate invoices	Must
FR-17	Expose results through SQL for ad hoc investigation	Must
FR-18	Log every run with row counts, rules applied, and elapsed time	Must
FR-19	Provide a dashboard summarising the current filing position	Could
FR-20	Evaluate AI generated rule proposals before a human approves them	Could

6. Non-functional requirements

ID	Requirement	Target
NFR-01	Full pipeline run on a month of data	Under 5 minutes on a laptop
NFR-02	Deterministic. Same input and config produce identical output	Exact
NFR-03	Idempotent. Re-running does not duplicate or corrupt results	Exact
NFR-04	Automated test coverage of tax calculation and rule resolution	90%+
NFR-05	Every rule change is version controlled and reviewable	100%
NFR-06	No personally identifying data in logs	100%
NFR-07	Runs locally with free and open source tooling only	Required

7. Assumptions

1. Source extracts are delivered daily as CSV. Real time is out of scope.
2. Jurisdiction is determined by the customer's billing address. Meridian

accepts this simplification for the pilot.

1. United States sales tax is modelled at state level only. County and city

precision is out of scope for the pilot and is a known limitation.

1. Currency conversion uses a monthly average rate, supplied by Finance.
2. Meridian's Tax team owns the rule configuration. Engineering owns the engine.
3. Historical filings will not be restated as part of this engagement.

8. Risks

Risk	Impact	Mitigation
Rule configuration becomes as complex as code	High	Decision table format, mandatory review, versioning
Tax SME and engineer interpret a rule differently	High	Golden dataset agreed and signed off before build
Source system changes a field without notice	Medium	Schema validation on ingest, fail loudly
Exception list too large to action	Medium	Severity and materiality thresholds, ranked by value
Over-reliance on AI generated rules	High	Deterministic validation and mandatory human approval
State level US modelling misses local rates	Medium	Documented limitation, flagged on every US output

9. Acceptance criteria

The pilot is accepted when all of the following hold on one full month.

1. Every transaction in the source extracts appears exactly once in the

canonical model, or on the rejection report with a reason.

1. 100% of golden dataset transactions produce the expected tax code, rate and

tax amount.

1. Reconciliation classifies 100% of variances into a named category. No

variance is left as UNKNOWN.

1. All twelve mandatory data quality checks execute and report.
2. Every calculated row carries a rule id, rule version and timestamp.
3. A Tax Analyst can change a rate in configuration and see the effect, with no

Python change, and the regression suite proves nothing else moved.

1. The pipeline reruns end to end and produces byte-identical output.
2. An auditor can select any filed figure and trace it to the source row and

the rule that produced it, in under two minutes.

Open questions for the Tax SME

Deliberately unanswered. You will resolve these in the Day 3 and Day 7 requirements workshop simulations.

1. When a customer's exemption certificate expires mid-month, is the whole

month taxable or only transactions after the expiry date?

1. For a UK sale shipped to Northern Ireland, which rules apply?
2. If a credit note reverses a transaction taxed at an old rate, does the

credit use the original rate or the current one?

1. What is the materiality threshold below which a variance is not worth

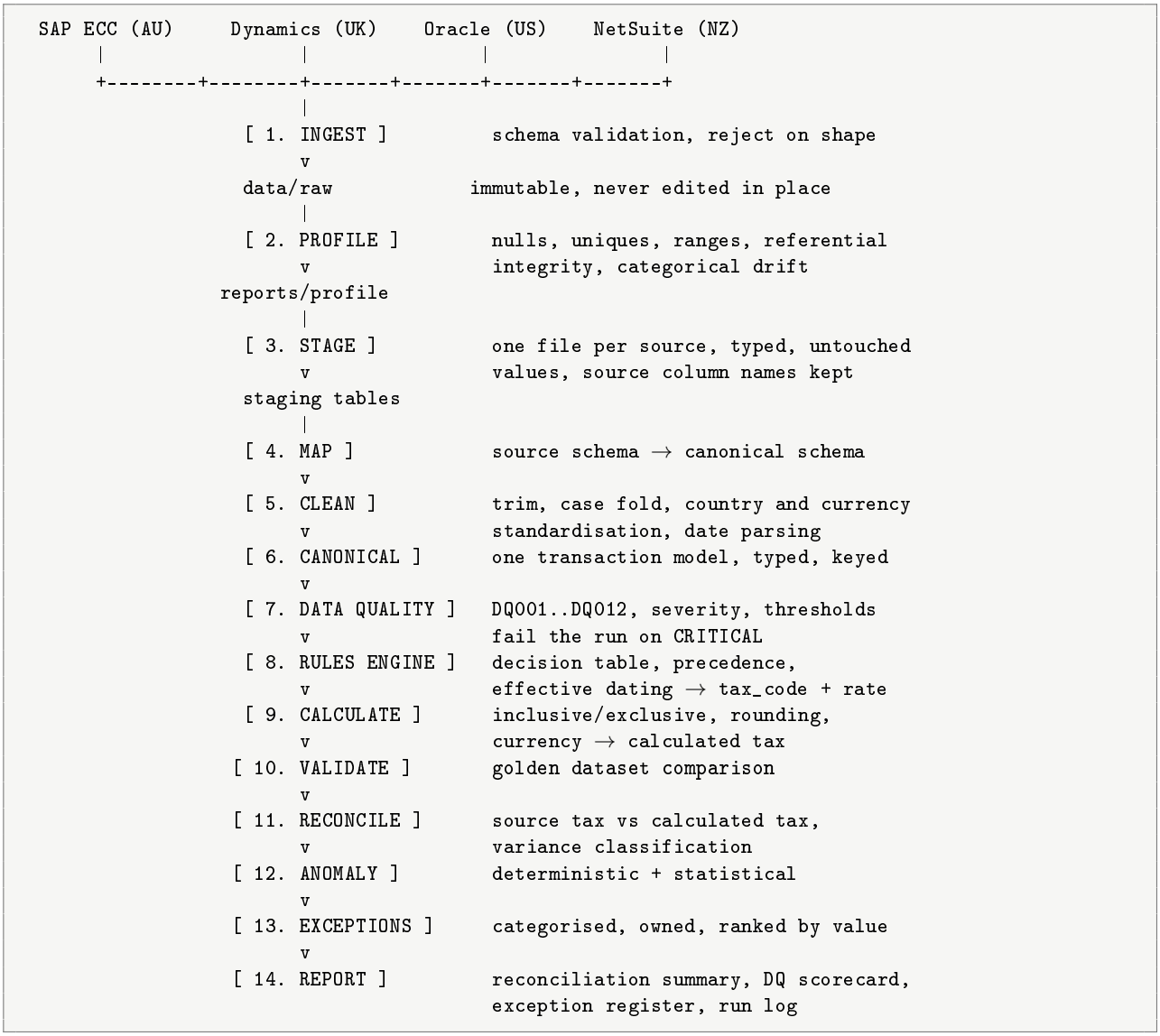
investigating?

1. Freight is currently treated as its own category. Should it follow the tax

treatment of the goods it delivers?

TaxTrace – Architecture

Pipeline



Every stage is a pure function of its input plus configuration. That is what makes the pipeline **idempotent**: run it twice, get the same answer.

Layer contract

Layer	Owns	Must not
raw	Bytes as delivered	Be edited, ever
staging	Types, one row per source row	Change business meaning
canonical	One schema, standardised values	Apply tax logic
rules	Which rule applies	Do arithmetic

Layer	Owns	Must not
calculated	The arithmetic	Decide which rule applied
reconciled	Comparison and classification	Change either input

INTERVIEW ALERT. Being able to say *why* those boundaries exist is worth more than being able to write any single stage. The reason is blast radius: when a number is wrong you need to know which layer to look in.

Traceability model

Every calculated row carries:

```
transaction_id, rule_id, rule_version, tax_code, tax_rate,
taxable_amount, calculated_tax, calculation_timestamp, engine_version, status
```

That tuple is the audit answer. Given a filed figure, you can name the rule, the version of that rule, and the source row.

Where cloud technology would sit in a real build

The local implementation is deliberately free. This is how it would map at a real client, and it is worth being able to describe in an interview.

Local here	Enterprise equivalent	What it would add
CSV in data/raw	Azure Data Lake Storage Gen2	Durable, versioned, access controlled landing zone
Python scripts	Azure Data Factory or Airflow	Scheduling, retries, dependencies, alerting, backfill
DuckDB	Azure SQL, Synapse or Snowflake	Concurrency, security, scale beyond one machine
Local pytest	Azure DevOps or GitHub Actions	Gate every merge on the regression suite
Local run log	Application Insights or Datadog	Alerting when exception rate moves
YAML in git	Same, plus an approval workflow	Segregation of duties on rule changes
Manual run	Databricks job or Function App	Elastic compute for month end peaks

Be honest in the interview. Say "I built this locally with DuckDB, and here is exactly how it would map onto Azure Data Factory and Synapse." That is a much stronger answer than pretending to Azure experience you do not have.

ETL versus ELT, in this project's terms

ETL transforms before loading. You would clean and apply tax rules in Python, then load only the finished result into the warehouse.

ELT loads raw first, then transforms inside the warehouse. TaxTrace is closer to ELT: raw lands untouched, staging and canonical models are built with SQL in DuckDB, and the transformations

are versioned in git.

ELT wins here for one specific reason that matters to tax: **you can always re-derive the answer from raw.** When a rate is discovered to be wrong six months later, ELT lets you reprocess history. ETL that discarded the raw input cannot.

Seven day plan

Aggressive but survivable. Roughly 6 hours a day. If a day runs over, cut the stretch task, never the checkpoint.

Each day follows the same loop: **concept, your attempt, my review, production solution, commit.** You write the important pieces. I review and then show you what a production version looks like and why it differs from yours.

Day 1 – Messy data and profiling

Detail: `day01.md`

- Why profiling comes before cleaning
- Loud, medium and subtle faults
- Referential integrity, whitespace on join keys, null versus wrong
- **Build:** `profile_dataframe()` and a profile report over all five files
- **Checkpoint:** profile runs, findings written, branch committed
- **Then:** score your profiler against the fault log

Day 2 – SQL, the canonical model and source mapping

- DuckDB, loading CSV, why it beats pandas for joins at this size
- Canonical schema design, mapping four ERPs onto one model
- SELECT, WHERE, CASE, joins, GROUP BY, HAVING, CTEs
- Window functions: ROW_NUMBER, RANK, LAG, LEAD
- Duplicate detection, conditional aggregation, NULL semantics
- **Build:** staging and canonical tables, source mapping config
- **Practice:** 20 SQL questions on this data, you answer first
- **Benchmark:** pandas vs Polars vs DuckDB, measured not asserted

Day 3 – The rules engine

- Decision tables, precedence, effective dating, rule versioning
- Configuration versus code, and where that boundary should sit
- **Build:** `tax_rules.yaml` and the engine that evaluates it
- **Build:** the calculation spec, written *before* the code
- Tax-inclusive vs exclusive, rounding, currency, zero-rated vs exempt
- **Simulation:** requirements workshop, I play the Tax SME and give you a

vague requirement. You ask the questions.

Day 4 – Data quality, reconciliation, anomalies

- Twelve named DQ checks with severity, threshold and status
- Golden dataset: what it is, why it is signed off before build
- Reconciliation: MATCH, ROUNDING_DIFFERENCE, RATE_MISMATCH, DATA_ERROR,

RULE_ERROR, MISSING_DATA, DUPLICATE, UNKNOWN

- Anomaly detection: z-score, IQR, percentile, plus deterministic rules
- **Build:** DQ framework, reconciliation engine, exception register
- **Measure:** your precision and recall against the fault log

Day 5 – Testing and the production incident

- pytest structure, fixtures, parametrisation, coverage of business logic
- Regression testing: change one rule, prove nothing else moved
- Property-based thinking on tax invariants
- **Incident:** I give you symptoms only. You investigate with SQL and logs,

find root cause, fix, rerun regression, verify, and write the incident report. I do not tell you the answer.

Day 6 – Git, CI/CD, docs, AI evaluation

- Feature branches, meaningful commits, a PR you would approve
- GitHub Actions: lint, test, coverage, fail on critical
- Logging and observability, what to log and what never to log
- Data dictionary and technical decisions record
- **Build:** AI evaluation harness. Natural language rule to structured

proposal to deterministic validation to human approval. PASS, FAIL, REQUIRE_HUMAN_REVIEW. Hallucinations, ambiguity, precedence, safety.

Day 7 – Interview

- Full mock interview, graded 1 to 5 per answer
- Python debugging simulations, deliberately broken code
- SQL investigation simulations, symptoms only
- Code review simulations, mediocre code to critique
- Behavioural questions using your real Stripe examples
- **Produce:** resume bullets, 30 second, 60 second and 2 minute explanations,

and the final knowledge matrix

Priorities if you fall behind

Do these no matter what: **Day 2 SQL**, **Day 3 rules engine**, **Day 4 reconciliation**, Day 5 incident**. Those four are the job.

Drop first: the Streamlit dashboard, the Polars benchmark, the AI harness beyond a working skeleton.

Day 1 – Messy data, profiling, and learning to distrust a file

Time budget: 5 to 7 hours **Theme:** You cannot fix what you have not measured.

Learning objectives

By the end of today you should be able to answer these out loud, without notes.

1. What is data profiling and why does it happen *before* cleaning?
2. What is referential integrity and how do you test it with a join?
3. Why does whitespace on a join key cost more than a null does?
4. What is the difference between a column being *null* and being *wrong*?
5. Why is a fault that is easy to see less dangerous than one that is not?

What you have been given

Run this if you have not already.

```
cd "C:\Monash University\Projects\Tax Project"
python scripts/generate_data.py
```

That writes:

```
data/raw/erp_sap_au_transactions.csv        18,182 rows   SAP field names
data/raw/erp_dynamics_uk_transactions.csv   14,061 rows   Dynamics field names
data/raw/erp_oracle_us_transactions.csv     12,076 rows   Oracle field names
data/raw/erp_netsuite_nz_transactions.csv    5,981 rows   NetSuite field names
data/raw/customers.csv                       4,000 rows
data/raw/vendors.csv                           800 rows
data/reference/*.csv                        jurisdictions, codes, rates, categories
data/golden/golden_dataset.csv                 250 rows   verified expectations
data/_answer_key/                           DO NOT OPEN YET
```

The answer key is real. `data/_answer_key/fault_log.csv` lists every fault that was deliberately injected, by transaction id. You will use it at the end of Day 1 to score yourself. Opening it before you have written your profiler is cheating yourself out of the exercise, and the whole point of this project is that you can measure your own recall.

12,468 faults were injected into 50,300 delivered rows. Some are loud. Some are designed to be invisible on a summary and only findable by reconciliation.

Concept 1 – Why profile first

DELOITTE RELEVANCE. On a real engagement you get a client extract on a Friday and you are expected to say something intelligent about it on Monday. The first thing you produce is a profile, not a fix. A profile tells you whether the file is even usable, gives you the questions to ask the client, and becomes the baseline you compare next month's extract against.

A profile answers, for every column:

- How much of it is missing?
- How many distinct values are there, and is that plausible?
- What are the extremes?
- What type is it *really*, as opposed to what it claims to be?
- Do the values that should point at another table actually resolve?

PRODUCTION ALERT. The most expensive profiling finding is usually *cardinality drift*: last month `currency_code` had 4 distinct values, this month it has 11. Nobody told you. A source system started sending `A$`.

Concept 2 – Loud, medium and subtle faults

Tier	Example in this dataset	Found by
LOUD	`customer_id` is null, date reads `TBC`	A profile
MEDIUM	Tax code `ZZ-UNKNOWN` not in the reference table	A join
SUBTLE	Tax understated by 0.8 per cent	Reconciliation only

Today you build the tool that catches the LOUD tier and most of the MEDIUM tier. The SUBTLE tier is Day 4, and it is where the real money is.

INTERVIEW ALERT. "How would you approach a dataset you have never seen?" The structured answer is: profile, then check referential integrity, then check business rules, then reconcile against an independent source. In that order, because each stage assumes the previous one passed.

Concept 3 – The whitespace problem

1,497 rows in this dataset have a `customer_id` like `" CUST001186 "`.

A null customer id is *loud*. Your profile shows 600 nulls and you go and ask.

A whitespace-padded customer id is *silent*. It is not null. It looks correct in Excel. It joins to nothing, so the customer's exemption status is never found, so the transaction is taxed at the standard rate, so your filing is wrong and nothing anywhere reported an error.

That single idea is most of what separates a data analyst from a data engineer.

Your first exercise

Build the profiling framework. This is yours to write, not mine.

Task

Create `src/taxtrace/profiling/profiler.py` with one public function:

```
def profile_dataframe(df: pd.DataFrame, table_name: str) → pd.DataFrame:
    """Return one row per column describing that column."""
```

The returned frame must have one row per column of `df` and these columns:

Output column	Meaning
`table_name`	The name you passed in
`column_name`	Name of the column being profiled
`dtype`	Pandas dtype as a string
`row_count`	Total rows in the table
`null_count`	Nulls, including empty strings and whitespace-only strings
`null_pct`	Null count as a percentage, rounded to 2dp
`distinct_count`	Number of distinct non-null values
`distinct_pct`	Distinct as a percentage of non-null, rounded to 2dp
`min_value`	Minimum, as a string. Blank if not sensible
`max_value`	Maximum, as a string. Blank if not sensible
`mean_value`	Mean for numeric columns only, else blank
`top_value`	Most common non-null value
`top_value_pct`	Share of non-null rows holding that value, 2dp
`has_leading_trailing_space`	`True` if any value has surrounding whitespace
`sample_values`	Three example non-null values, pipe separated

Rules

1. It must work on a column of **any** dtype without raising. Every column in

these files is read as text by default, so do not assume numbers.

1. Treat `""`, `" "` and `"N/A"` as null for `null_count`. Real profilers have

a configurable null-token list. Make yours a parameter with a default.

1. Do not mutate the input frame. Profiling that changes the thing it profiles

is a bug you will only find at 2am.

1. No hardcoded column names anywhere.

Then run it

Write `scripts/run_profile.py` that profiles all four ERP extracts plus `customers.csv`, concatenates the results, and writes `reports/profiling/data_profile.csv`.

Read every file with `dtype=str` so pandas does not silently coerce anything. That is deliberate. You want to see the raw text.

Questions to answer in writing when you are done

Put your answers in `docs/day01_findings.md`. Three or four sentences each.

1. Which column across all four files has the highest null percentage, and what

would you ask the client about it?

1. `LAND1` in the SAP file and `CountryRegionCode` in the Dynamics file both

hold country. How many distinct values does each have? What does that tell you about standardising them?

1. Find one column where `distinct_count` equals `row_count`. What does that

tell you about the column, and is it true in every file?

1. Pick the two faults you think are most dangerous and say why. Not the most

common. The most dangerous.

Stretch, only if you have time

Add a `-html` flag that also writes `reports/profiling/data_quality_report.html`. Plain HTML from `DataFrame.to_html` is fine. Do not install anything.

Checkpoint

You are done with Day 1 when:

- ☐ `profile_dataframe` runs on all five files without raising
- ☐ `reports/profiling/data_profile.csv` exists and has one row per column per table
- ☐ You have written `docs/day01_findings.md`
- ☐ You can explain, out loud, why whitespace on a key is worse than a null
- ☐ You have committed your work on a branch called `feature/day1-profiling`

Do not open the answer key until the checkpoint is complete. When it is, tell me and I will review your implementation line by line, show you what a production version looks like, and only then we will score your profiler against the fault log together.

Hints, if you get stuck

Read these one at a time, not all at once.

Hint 1, structure
Loop over `df.columns`, build a dict per column, then `pd.DataFrame(rows)` at the end. Do not try to do it with a single clever aggregation.

Hint 2, the null definition
`s.isna()` catches real nulls. For the token nulls you need something like `s.astype(str).str.strip().isin(null_tokens)`. Combine with `|`.

Hint 3, min and max on text columns
Try `pd.to_numeric(s, errors="coerce")`. If most values convert, treat the column as numeric for min, max and mean. If not, fall back to string min and max, which is still useful for spotting a stray value.

Hint 4, whitespace detection
```
(s.astype(str) != s.astype(str).str.strip()).any()
```